포팅 매뉴얼

DocQ 서비스 포팅 매뉴얼 (SSAFY)

1. 문서 개요

1.1 목적

본 문서는 DocQ 프로젝트를 **GitLab 소스 클론 이후 빌드 및 배포**할 수 있도록, 필요한 실행 환경, 설정 값, 외부 서비스, 배포 절차를 정리한 포팅 매뉴얼이다.

1.2 대상

- SSAFY 프로젝트 심사자
- 프로젝트 배포 및 운영 담당자

---

2. 시스템 구성 개요

2.1 아키텍처 구성

```
Client
 └─ HTTPS
     ├─ Nginx (SSL + Reverse Proxy)
     │    ├─ /api         → Backend (8081)
     │    ├─ /ws-docq     → WebSocket (8081)
     │    ├─ /ai          → AI Service (8000)
     │    └─ /            → Frontend
     │
     ├─ Backend (Spring Boot)
     │    └─ MySQL (AWS RDS)
     │
```

```
      ├─ AI Service (FastAPI)
      │    └─ Redis (ai-redis)
      │
      └─ Redis (backend)
```

2.2 설계 특징

- Backend / AI Redis 분리
- Nginx를 통한 SSL 종단 및 Reverse Proxy
- 대용량 PDF 업로드 고려 (40MB)
- AI 비동기 처리 구조

---

3. 사용 제품 및 버전

구분	제품	버전	용도
JVM	OpenJDK	17	Backend 실행
WAS	Apache Tomcat	Embedded	Spring Boot 내장
Web Server	Nginx	Stable 최신	Reverse Proxy, SSL
Frontend Runtime	Node.js	^20.19.0	Vite 빌드
AI Runtime	Python	3.11	FastAPI
Cache	Redis	7.x	Backend / AI
IDE	IntelliJ IDEA	2023.x	Backend 개발
IDE	VS Code	최신	Frontend / AI 개발

---

4. 디렉토리 구조

```
docq-frontend/
docq_backend/
docq-ai/
deploy/
 ├─ compose/
 │   ├─ docker-compose.yml
```

```
  │   └─ .env
  └─ nginx/
      ├─ conf.d/default.conf
      └─ common.conf
docs/
```

---

5. 환경변수 정의

> 보안상 민감한 값(API Key, DB 계정, 내부 IP)은 실제 값 대신 식별자 형태로 기재하였다.

5.1 Backend (`deploy/compose/.env`)

```
EC2_HOST_IP=<EC2_PRIVATE_IP>

SPRING_PROFILES_ACTIVE=prod

DB_HOST=<RDS_ENDPOINT>
DB_PORT=<DB_PORT>
DB_NAME=docq
DB_USER=<DB_USERNAME>
DB_PASS=<DB_PASSWORD>
```

5.2 AI Service (`docq-ai/.env`)

```
OPENAI_API_KEY=<OPENAI_API_KEY>
OPENAI_BASE_URL=https://gms.ssafy.io/gmsapi/api.openai.com/v1

MINERU_API_URL=http://localhost:18000/file_parse
```

5.3 Frontend (`docq-frontend/.env`)

```
VITE_API_BASE_URL=/api
VITE_WS_URL=/ws-docq
```

---

6. Backend 설정 파일

6.1 `application.properties`

```
spring.profiles.default=local

spring.application.name=docq_backend
server.port=8081

# Timezone
spring.jpa.properties.hibernate.jdbc.time_zone=Asia/Seoul

# File Size Limit
spring.servlet.multipart.max-file-size=50MB
spring.servlet.multipart.max-request-size=50MB

# Redis
spring.data.redis.host=${REDIS_HOST:localhost}
spring.data.redis.port=${REDIS_PORT:6379}

# Game event poller
game.events.poll-interval-ms=200

# Datasource
spring.datasource.url=jdbc:mysql://${DB_HOST}:${DB_PORT}/${DB
_NAME}?useSSL=false&serverTimezone=Asia/Seoul
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASS}

# JPA
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.open-in-view=false
```

> 실제 DB 접속 정보는 환경변수를 통해 주입된다.

---

7. 로컬 실행 가이드

7.1 Frontend

```
cd docq-frontend
npm install
npm run dev
```

- 접속: `http://localhost:5173`

7.2 Backend

```
cd docq_backend
./gradlew bootRun
```

- 접속: `http://localhost:8081`

7.3 AI Service

```
cd docq-ai
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

- 접속: `http://localhost:8000`

7.4 Redis (로컬)

```
docker run -d -p 6379:6379 redis:7-alpine
```

---

8. 서버 배포 가이드

8.1 사전 준비

- AWS EC2 (Linux)
- Docker / Docker Compose 설치
- `.env` 파일 구성
- SSL 인증서 (Let's Encrypt)

8.2 배포 실행

```
cd deploy/compose
docker compose up -d --build
```

8-3. EC2 최초 배포 단계

1. **EC2 접속**

- `ssh -i <KEY.pem> ubuntu@<EC2_HOST>`

1. **Docker/Compose 설치**

- Docker 설치 및 권한 설정 (README/기존 절차 참고)
- 설치 확인: `docker -v`, `docker compose version`

1. **프로젝트 클론**

- `git clone <REPO_URL>`
- `cd <REPO_DIR>/deploy/compose`

1. **환경변수 파일 준비**

- `deploy/compose/.env` 작성
- `docq-ai/.env` 작성
- 필요 값은 문서 5장 참조

1. **SSL 인증서 발급**

- Let's Encrypt 인증서 발급 후 `/etc/letsencrypt` 경로 확인
- Nginx 설정에서 도메인/경로 적용

1. **컨테이너 빌드/기동**

- `docker compose up -d --build`

1. **서비스 확인**

- `docker compose ps`
- `docker compose logs -f backend`
- 접속 확인: `https://<도메인>`

8-4. Docker Compose 파일 (복사/붙여넣기)

`deploy/compose/docker-compose.yml`

```
services:
  nginx:
    image: nginx:latest
    container_name: nginx
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - /home/ubuntu/jenkins-data/workspace/cicd_master/deplo
y/nginx/conf.d:/etc/nginx/conf.d:ro
      - /home/ubuntu/jenkins-data/workspace/cicd_master/deplo
y/nginx/common.conf:/etc/nginx/common.conf:ro
      - /etc/letsencrypt:/etc/letsencrypt:ro
    depends_on:
      - frontend
      - backend
    networks:
      - docq-network

  backend:
    build: ../../docq_backend
    restart: unless-stopped
```

```
    env_file:
      - .env
    environment:
      - SPRING_PROFILES_ACTIVE=prod
      - ai.url=http://ai-service:8000
      - REDIS_HOST=redis
      - REDIS_PORT=6379
    depends_on:
      - redis
    networks:
      - docq-network

  redis:
    image: redis:7-alpine
    container_name: redis
    restart: unless-stopped
    command: redis-server --appendonly yes
    volumes:
      - redis-data:/data
    networks:
      - docq-network

  frontend:
    build: ../../docq-frontend
    restart: unless-stopped
    networks:
      - docq-network

  ai-redis:
    image: redis:7-alpine
    container_name: ai-redis
    restart: unless-stopped
    # appendonly 없음 = 재시작 시 데이터 초기화
    networks:
      - docq-network
```

```
  ai-service:
    build: ../../docq-ai
    restart: unless-stopped
    env_file:
      - ../../docq-ai/.env
    environment:
      - PYTHONUNBUFFERED=1
      - MINERU_HOST=${EC2_HOST_IP}
      - REDIS_HOST=ai-redis
      - REDIS_PORT=6379
      - BACKEND_CALLBACK_URL=http://backend:8081/ai/callback
    depends_on:
      - ai-redis
    networks:
      - docq-network

  # --- 모니터링 서비스 추가 ---
  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    restart: unless-stopped
    volumes:
      - /home/ubuntu/jenkins-data/workspace/cicd_master/deplo
y/compose/prometheus/prometheus.yml:/etc/prometheus/prometheu
s.yml
      - prometheus-data:/prometheus
    ports:
      - "127.0.0.1:9090:9090" # SSH 터널링용 로컬 바인딩
    networks:
      - docq-network

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    restart: unless-stopped
    volumes:
```

```
      - grafana-data:/var/lib/grafana
    ports:
      - "127.0.0.1:3000:3000" # SSH 터널링용 로컬 바인딩
    networks:
      - docq-network

networks:
  docq-network:
    driver: bridge

volumes:
  redis-data:
  prometheus-data: # 데이터 보존용
  grafana-data: # 데이터 보존용
```

8-5. Dockerfile (복사/붙여넣기)

`docq_backend/Dockerfile`

```
# Build stage
FROM gradle:8.5-jdk17 AS build
WORKDIR /app

# Copy Gradle files
COPY build.gradle settings.gradle ./
COPY gradle ./gradle

# Download dependencies (cached layer)
RUN gradle dependencies --no-daemon || true

# Copy source code
COPY src ./src

# Build application
RUN gradle clean build --no-daemon -x test
```

```
# Runtime stage
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app

# Set timezone to match server (KST)
ENV TZ=Asia/Seoul
RUN apk add --no-cache tzdata \
    && cp /usr/share/zoneinfo/Asia/Seoul /etc/localtime \
    && echo "Asia/Seoul" > /etc/timezone

# Create non-root user
RUN addgroup -S spring && adduser -S spring -G spring
USER spring:spring

# Copy built JAR from build stage
COPY --from=build /app/build/libs/*.jar app.jar

EXPOSE 8081

ENTRYPOINT ["java", "-jar", "app.jar"]
```

`docq-ai/Dockerfile`

```
FROM python:3.11-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app

COPY requirements.txt .
RUN python -m pip install --upgrade pip && \
    pip install -r requirements.txt

COPY app ./app
```

```
EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--por
t", "8000"]
```

`docq-frontend/Dockerfile`

```
# Build stage
FROM node:20-alpine AS build
WORKDIR /app

# Copy package files
COPY package*.json ./

# Install dependencies
RUN npm ci

# Copy source code
COPY . .

# Build application
RUN npm run build

# Production stage
FROM nginx:alpine
WORKDIR /usr/share/nginx/html

# Remove default nginx static assets
RUN rm -rf ./*

# Copy built files from build stage
COPY --from=build /app/dist .

# Copy nginx configuration (optional, if custom config neede
d)
```

```
# 기본 nginx 설정 사용 (docker-compose에서 볼륨 마운트로 오버라이드 가능)
COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

8-6. UFW 방화벽 설정 (EC2)

```
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw allow 18000/tcp
sudo ufw allow 18001/tcp
sudo ufw allow 18002/tcp
sudo ufw enable
sudo ufw status
```

---

9. Nginx 설정

9.1 라우팅 구조

경로	대상
`/`	Frontend
`/api/`	Backend
`/ws-docq`	WebSocket
`/ai/`	AI Service

9.2 주요 설정

- `client_max_body_size 40m`
- `proxy_read_timeout 300s`

- SSL 인증서 경로: `deploy/nginx/conf.d/default.conf`

9.3 Nginx 설정 파일 (복사/붙여넣기)

`deploy/nginx/conf.d/default.conf`

```
server {
    listen 80;
    server_name i14d209.p.ssafy.io docq.site;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name i14d209.p.ssafy.io;

    ssl_certificate     /etc/letsencrypt/live/i14d209.p.ssaf
y.io/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/i14d209.p.ssaf
y.io/privkey.pem;

    include /etc/nginx/common.conf;
}

server {
    listen 443 ssl;
    server_name docq.site;

    ssl_certificate     /etc/letsencrypt/live/docq.site/fullc
hain.pem;
    ssl_certificate_key /etc/letsencrypt/live/docq.site/privk
ey.pem;

    include /etc/nginx/common.conf;
}
```

`deploy/nginx/common.conf`

```
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_prefer_server_ciphers on;

    client_max_body_size 40m;

    location ~ /\.(git|env|svn|hg) {
        deny all;
        return 404;
    }

    location / {
        proxy_pass http://frontend:80;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forward
ed_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /api/ {
        proxy_pass http://backend:8081/;
        proxy_connect_timeout 300s;
        proxy_send_timeout    300s;
        proxy_read_timeout    300s;
        send_timeout          300s;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forward
ed_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /ws-docq {
        proxy_pass http://backend:8081/ws-docq;
        proxy_http_version 1.1;
```

```
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forward
ed_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /ai/ {
        rewrite ^/ai/(.*) /$1 break;
        proxy_pass http://ai-service;
        proxy_connect_timeout 300s;
        proxy_send_timeout    300s;
        proxy_read_timeout    300s;
        send_timeout          300s;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forward
ed_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
```

---

10. DB 접속 정보 및 ERD 연계

10.1 주요 DB 환경변수

```
DB_HOST
DB_PORT
DB_NAME
DB_USER
DB_PASS
```

10.2 관련 설정 파일 목록

- `application.properties`
- `application-local.properties`
- `application-prod.properties`
- `deploy/compose/.env`

---

11. 외부 서비스 정보

서비스	용도	비고
Upstage GMS	LLM 퀴즈 생성	OpenAI 호환
MinerU	PDF 파싱	선택
AWS RDS	MySQL DB	외부
Let’s Encrypt	SSL	HTTPS

---

12. 빌드 및 배포 절차 요약

1. GitLab Repository Clone
2. 환경변수(.env) 설정
3. Docker Compose 이미지 빌드
4. Nginx Reverse Proxy / SSL 설정
5. Backend / Frontend / AI 서비스 기동

---

13. 테스트 체크리스트

항목	상태
API 연동	정상
Redis 연결	정상
AI 콜백	정상
PDF 업로드	정상
HTTPS	정상
WebSocket	정상

14. 보안 관련 안내

```
보안상 민감한 정보(API Key, DB 계정, 내부 IP)는
제출 문서에서 실제 값을 제외하였으며,
서버 환경에서는 .env 또는 CI/CD Secret으로 관리한다.
```